

Data Structures and Algorithms

Lab 03 - Linked Lists

Instructor: Sidrat-ul-Muntha

READ IT FIRST

Prior to start solving the problems in this assignments, please give full concentration on following points.

1. WORKING – This is individual lab. If you are stuck in a problem contact your teacher, but, in mean time start doing next question (don't waste time).
 2. DEADLINE – One Week after assigned this.
 3. SUBMISSION – This assignment needs to be submitted in a soft copy.
 4. WHERE TO SUBMIT – Please visit your LMS.
 5. HOW TO SUBMIT – rename all .java files as task number and make .zip file, name it as 000-00-0000_assignment.zip, and submit .java file ONLY.
-

Task 1: Singly Linked List Class

Create a **singly linked list** in Java by defining a **List** interface, a **Node** class, and a **LinkedList** class that implements the interface.

In addition to the methods defined in the **List** interface, the **LinkedList** class must implement the **toString()** method.

The **toString()** method should return the linked list in the following format:

[size: the_size_of_the_list - item1, item2, item3,]

List Interface

```
interface List {  
    public boolean isEmpty();  
    // returns true if the list is empty, false otherwise
```

```
public int size();  
    // returns the number of nodes in the list  
public void add(Node n);  
    // adds a node at the end of the list  
public void add(int data, Node n);  
    // adds a node after the node containing the given data  
public void remove(int data);  
    // removes the first node containing the given data  
public void remove(Node n);  
    // removes the node whose data matches the given node  
public LinkedList Duplicate();  
    // creates and returns a true (independent) copy of the linked list  
public LinkedList DuplicateReverse();  
    // creates and returns a true (independent) duplicate of the list in reverse order  
}
```

Node Class

```
class Node {  
    public int data; // stores the data of the node  
    public Node next; // reference to the next node  
    // Constructor  
    public Node(int d) {  
    }  
}
```

LinkedList Class

```
class LinkedList implements List {  
    Node head; // reference to the first node
```

```
int size; // stores the number of nodes

// Constructor
LinkedList() {
    head = null;
    size = 0;
}

// Implement all methods defined in the List interface
}
```

Task 2: Linked List with Tail Reference

Modify your LinkedList implementation by adding a **tail** reference that points to the last node.

Task Requirements:

- Add a **tail** member to the LinkedList class.
- Update the constructor to initialize both head and tail.
- Modify the linked-list implementation to properly use the tail reference.
- Ensure that all linked-list operations continue to work correctly.
- Analyze whether adding a tail reference **increases the efficiency** of the existing methods.

LinkedList Class

```
class LinkedList implements List {
    Node head; // reference to the first node
    Node tail; // reference to the last node
    int size; // stores the number of nodes

    // Constructor
    LinkedList() {
        head = null;
        tail = null;
        size = 0;
    }
}
```

```
}
```

```
// Update the methods of Task 1 to incorporate the tail reference
```

```
}
```
